



US012386915B2

(12) **United States Patent**  
**Hulick, Jr. et al.**

(10) **Patent No.:** **US 12,386,915 B2**  
(45) **Date of Patent:** **Aug. 12, 2025**

(54) **DYNAMIC WEB CONTENT MANAGEMENT SYSTEM**

(71) Applicant: **Cisco Technology, Inc.**, San Jose, CA (US)

(72) Inventors: **Walter Theodore Hulick, Jr.**, Pearland, TX (US); **David John Zacks**, Vancouver (CA); **Thomas Szigeti**, Vancouver (CA)

(73) Assignee: **Cisco Technology, Inc.**, San Jose, CA (US)

(\*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

(21) Appl. No.: **17/845,116**

(22) Filed: **Jun. 21, 2022**

(65) **Prior Publication Data**

US 2023/0409662 A1 Dec. 21, 2023

(51) **Int. Cl.**  
**G06F 16/958** (2019.01)  
**H04L 67/02** (2022.01)

(52) **U.S. Cl.**  
CPC ..... **G06F 16/972** (2019.01); **H04L 67/02** (2013.01)

(58) **Field of Classification Search**  
None  
See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

9,418,170 B2 8/2016 Seolas et al.  
10,432,662 B2 10/2019 Sethi et al.

10,938,927 B2 3/2021 Crossley et al.  
10,956,575 B2 \* 3/2021 Harsany ..... G06F 21/575  
11,451,565 B2 \* 9/2022 Zhou ..... G06N 3/04  
2013/0014018 A1 \* 1/2013 Miner ..... G06Q 10/109  
715/736  
2014/0041048 A1 2/2014 Goodwin et al.  
2014/0280515 A1 \* 9/2014 Wei ..... H04L 67/565  
709/203  
2015/0088975 A1 \* 3/2015 Wei ..... G06F 9/54  
709/203  
2020/0366696 A1 \* 11/2020 Willis ..... H04L 63/1425  
2021/0216642 A1 \* 7/2021 Barraza Enciso .... G06F 21/577  
2021/0385245 A1 \* 12/2021 Melson ..... H04L 63/1433  
2022/0038468 A1 2/2022 Jeevagunta et al.  
2022/0256006 A1 \* 8/2022 Johannsen ..... H04L 67/025

#### OTHER PUBLICATIONS

Wesley Chai, "Google Analytics" Apr. 13, 2022, TechTarget.com (Year: 2022).\*

(Continued)

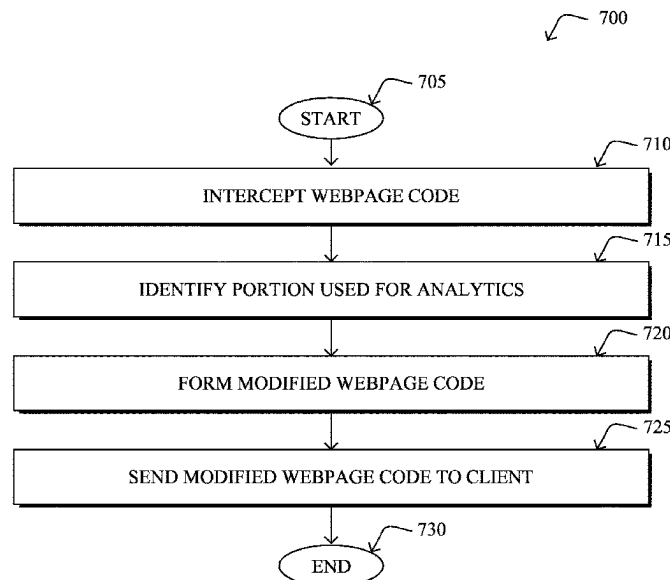
*Primary Examiner* — Ariel Mercado-Vargas

(74) *Attorney, Agent, or Firm* — BEHMKE  
INNOVATION GROUP LLC; James M. Behmke

#### (57) **ABSTRACT**

In one embodiment, an agent executed by a device intercepts webpage code for a website sent from an application server to a client of the website. The agent identifies a portion of the webpage code as being used for webpage analytics. The agent forms modified webpage code by disabling the portion of the webpage code, based on one or more performance metrics associated with the website. The agent sends the modified webpage code to the client of the website.

**19 Claims, 9 Drawing Sheets**



(56)

**References Cited****OTHER PUBLICATIONS**

Arsenault, Cody, "An Overview of Image Metadata—How It Affects Web Performance and Security", online: <https://www.keycdn.com/blog/image-metadata>, Apr. 10, 2018, accessed Apr. 28, 2022, 4 pages, proinity LLC.

Conzett, Lance, "How do I delete tags through Tag Manager?", online: <https://raven.zendesk.com/hc/en-us/articles/202279634-How-do-I-delete-tags-through-Tag-Manager->, Nov. 5, 2017, accessed Apr. 28, 2022, 2 pages, Raven Internet Marketing Tools.

"Eliminate the Render Blocking Effect of Analytics JavaScript", online: <https://pagespeedchecklist.com/defer-google-analytics>, accessed Apr. 28, 2022, 5 pages, PageSpeedChecklist.com.

"Web Page Meta Tags: What They Are and the Legal Issues Affecting Their Use", online: <https://www.stimmel-law.com/en/articles/web-page-meta-tags-what-they-are-and-legal-issues-affecting-their-use>, accessed Apr. 28, 2022, 4 pages, Stimmel, Stimmel & Roeser.

Gill, Tony, "Metadata and the Web", online: <https://www.getty.edu/publications/intrometadata/metadata-and-the-web/>, accessed Apr. 28, 2022, 24 pages, Getty.edu.

"<meta>: The metadata element", online: <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/meta>, accessed Apr. 28, 2022, 4 pages, Mozilla Corporation.

"Meta tags that Google understands", online: <https://developers.google.com/search/docs/advanced/crawling/special-tags>, Apr. 21, 2022, accessed Apr. 28, 2022, 4 pages, Google Developers.

Pham, Ha, "Why Google Analytics sucks and what you can do about it", online: <https://www.holistics.io/blog/why-google-analytics-sucks-and-what-you-can-do-about-it/>, Dec. 17, 2019, accessed Apr. 28, 2022, 4 pages, Holistics.io.

Peiris, Dave, "Reduce The Impact of Third-Party Code", online: <https://www.propellernet.co.uk/is-third-party-javascript-slowng-you-down/>, Jul. 9, 2020, accessed Apr. 28, 2022, 3 pages, Propellernet.

"Google Analytics Usage Statistics", online: <https://trends.builtwith.com/analytics/Google-Analytics>, accessed Apr. 28, 2022, 4 pages, BuiltWith Pty Ltd.

"Measuring the impact of Google Analytics", online: [https://smerity.com/cs205\\_ga/](https://smerity.com/cs205_ga/), accessed Apr. 28, 2022, 15 pages, github.com.

\* cited by examiner

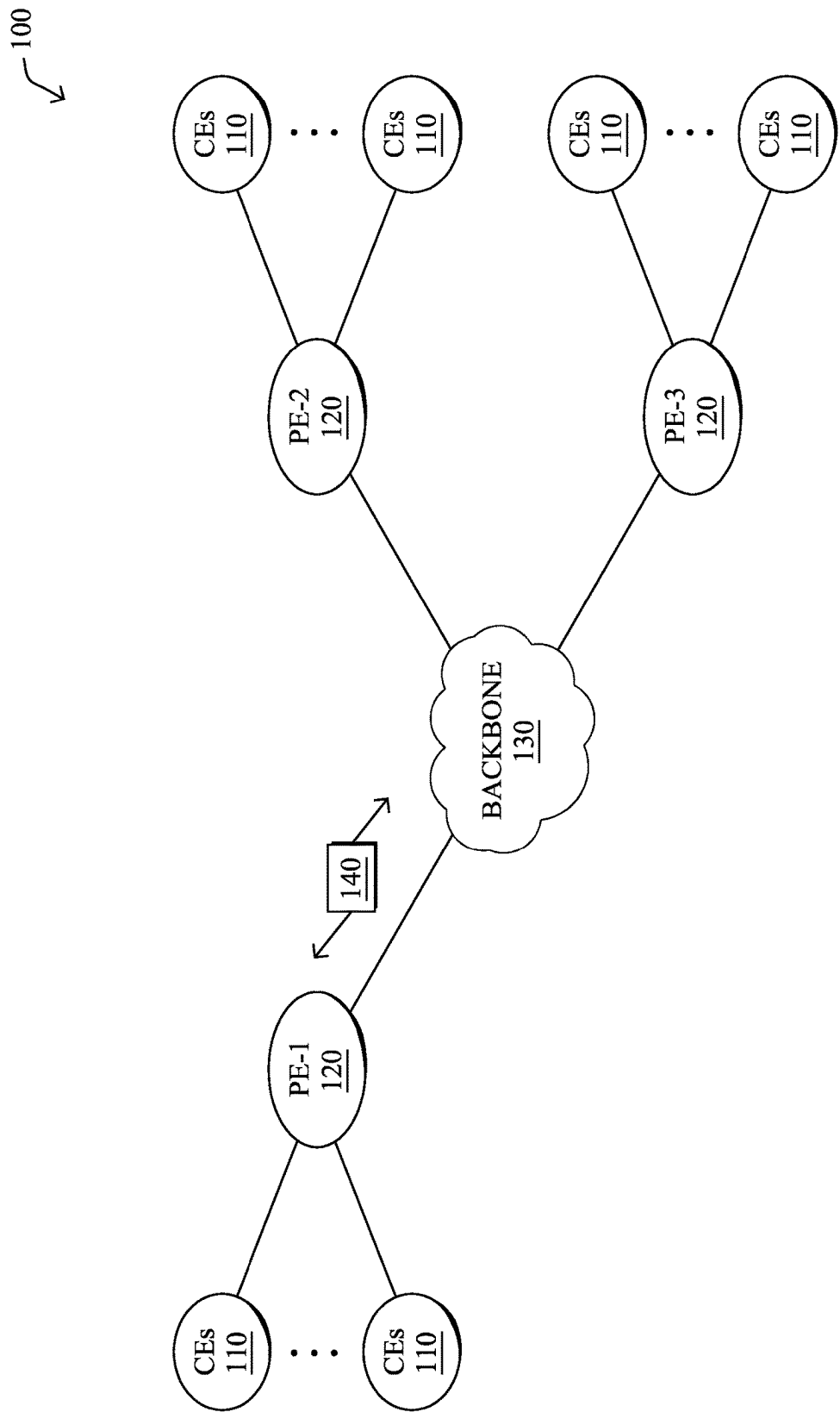


FIG. 1A

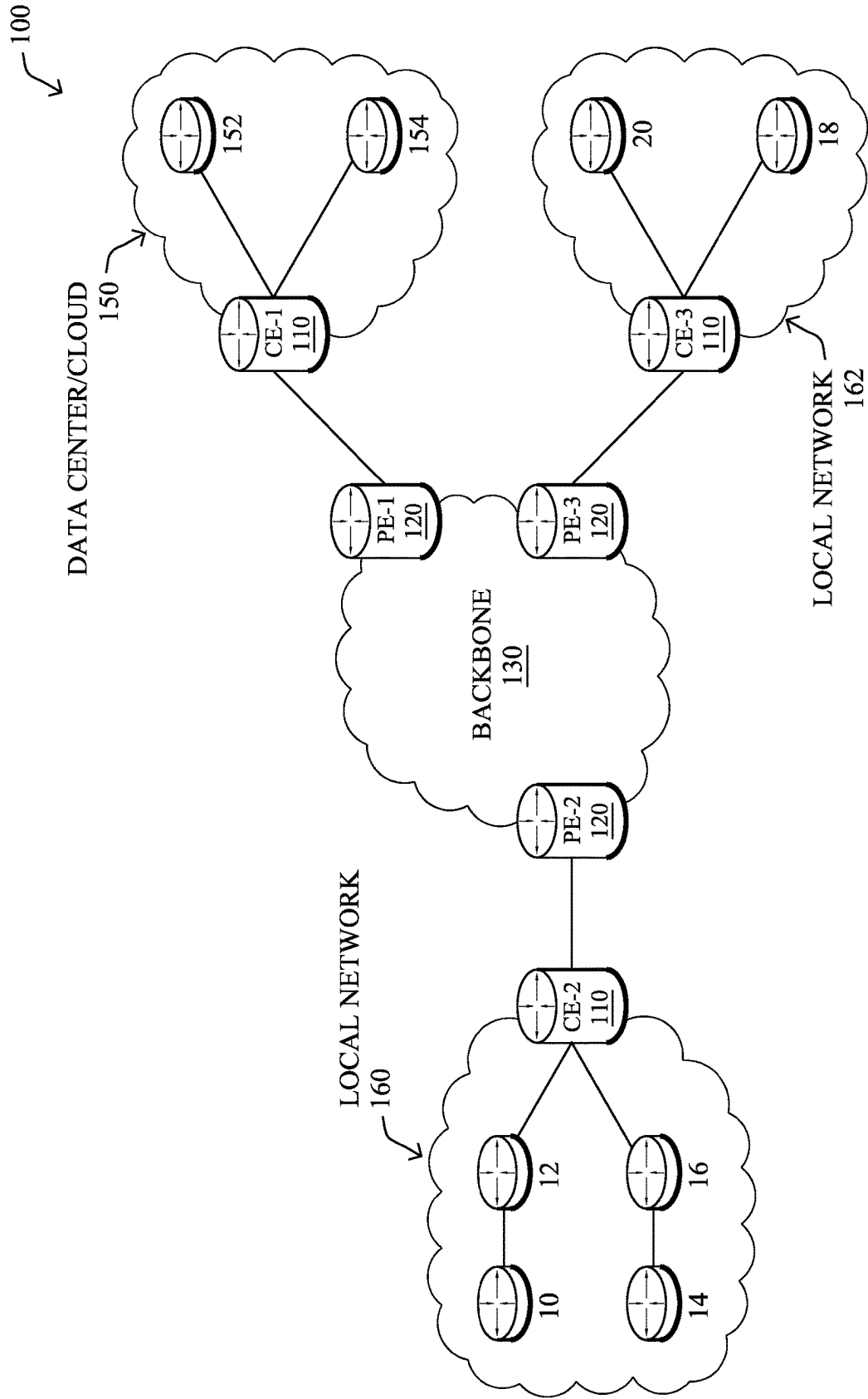


FIG. 1B

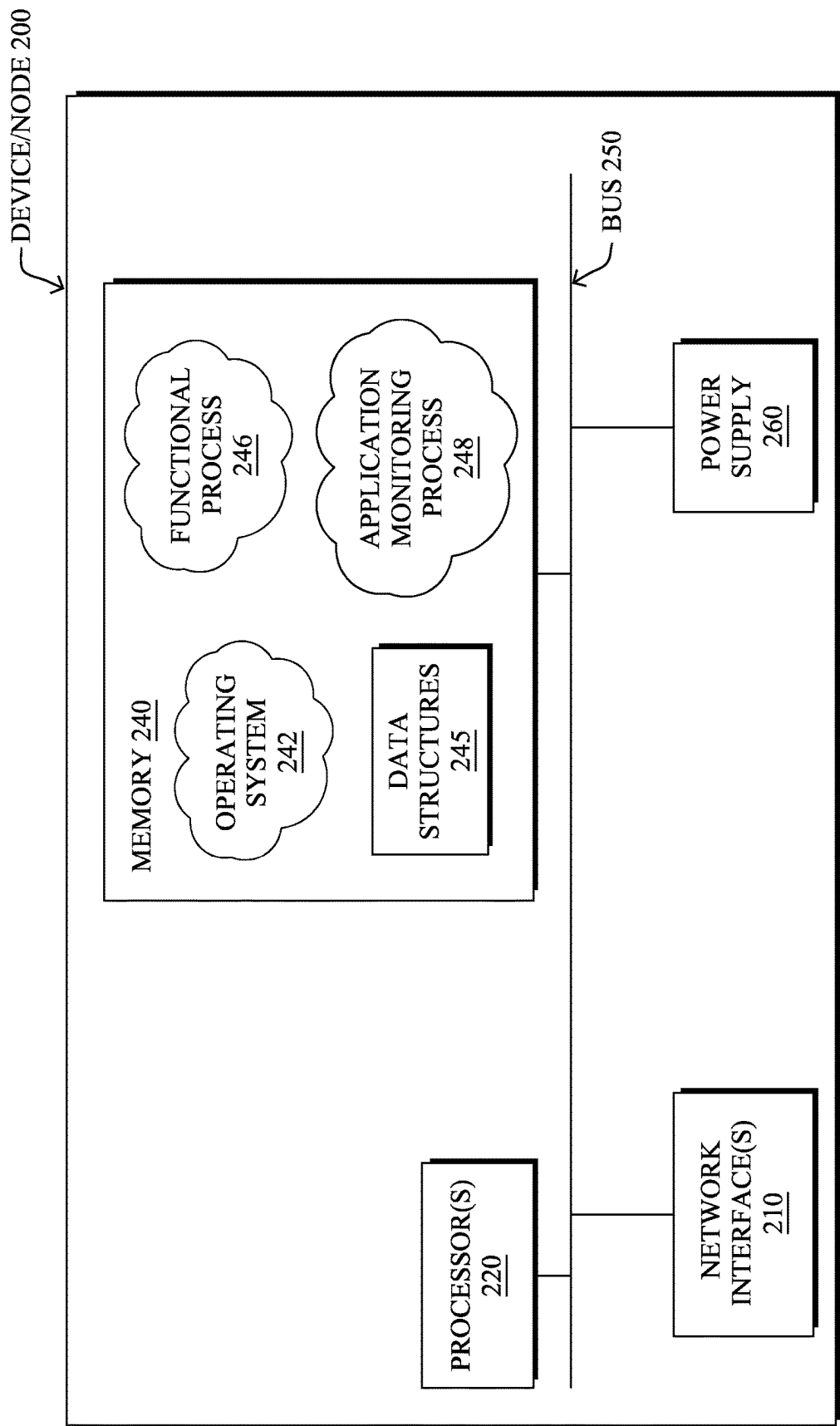


FIG. 2

300 ↘

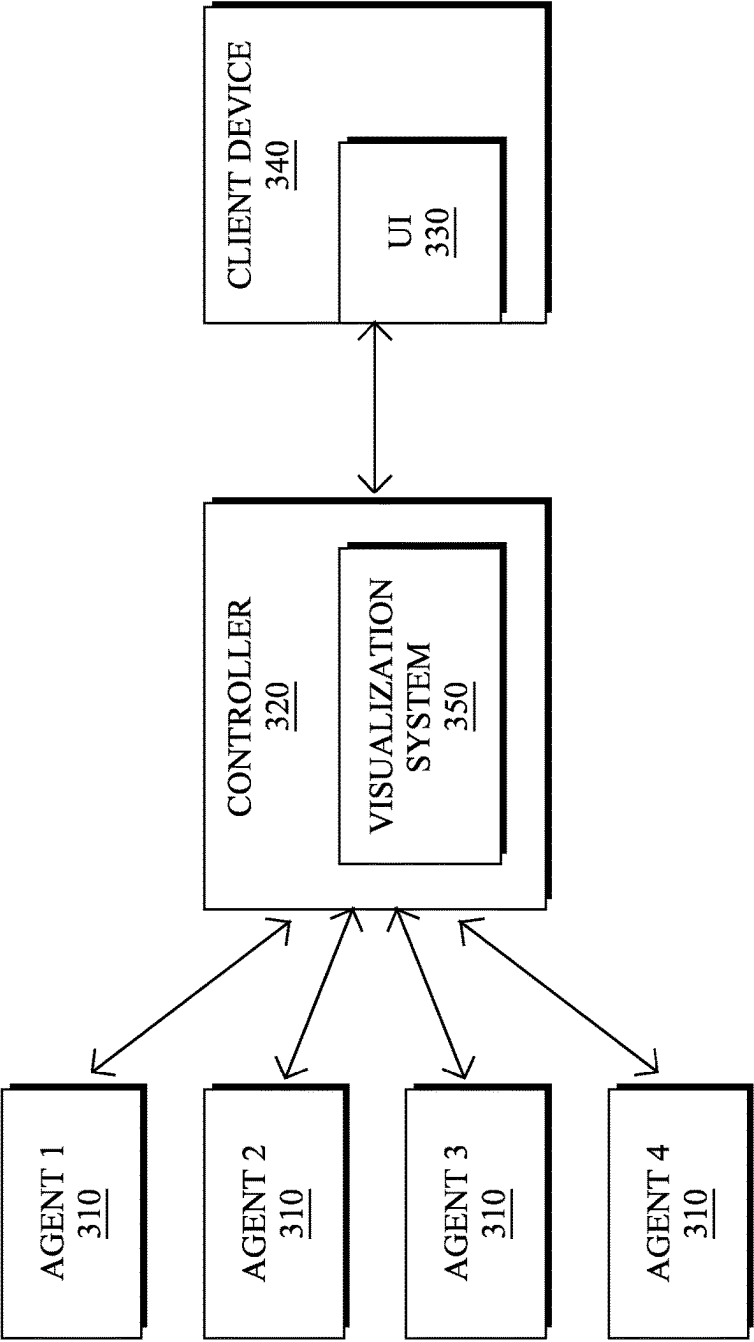


FIG. 3

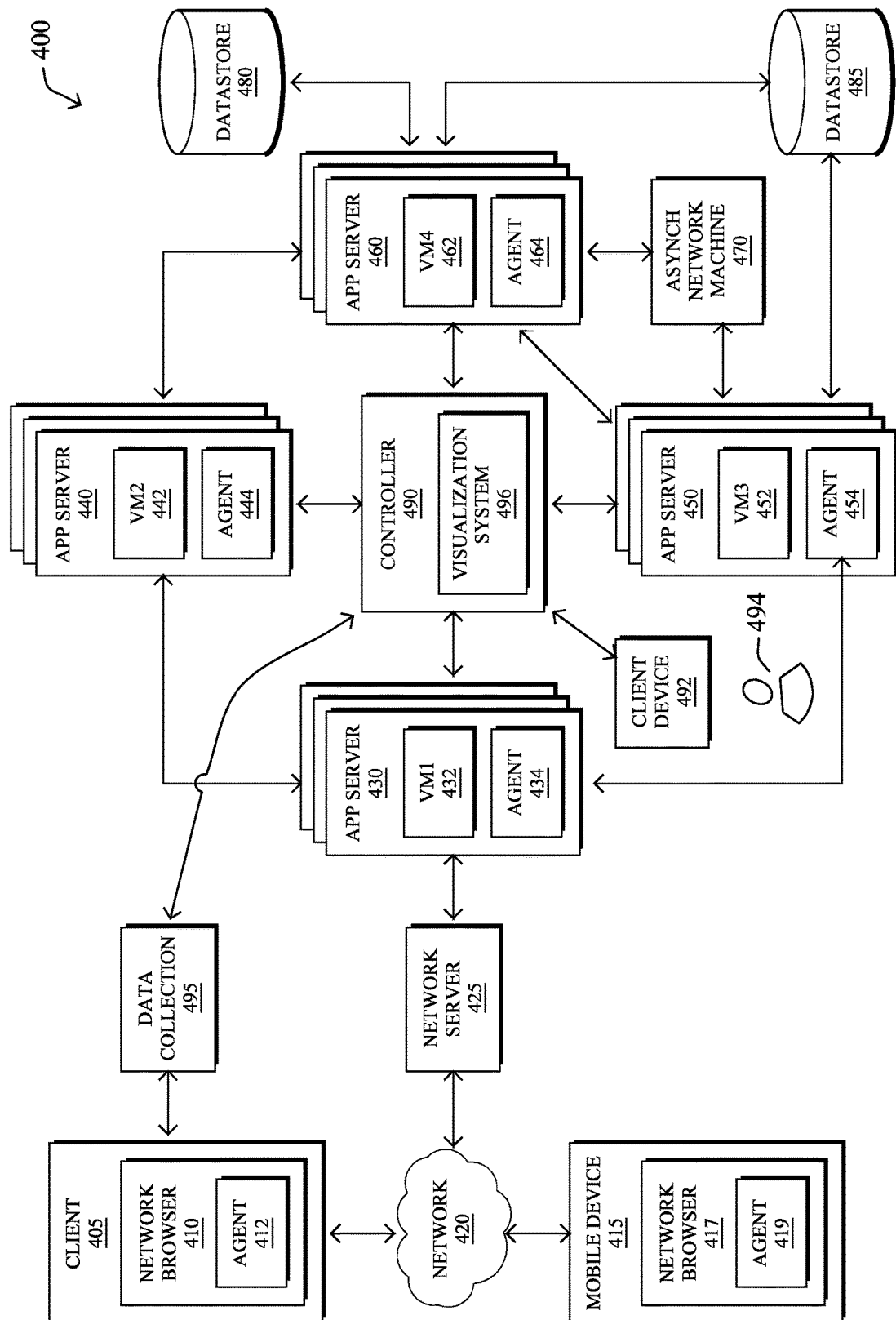


FIG. 4

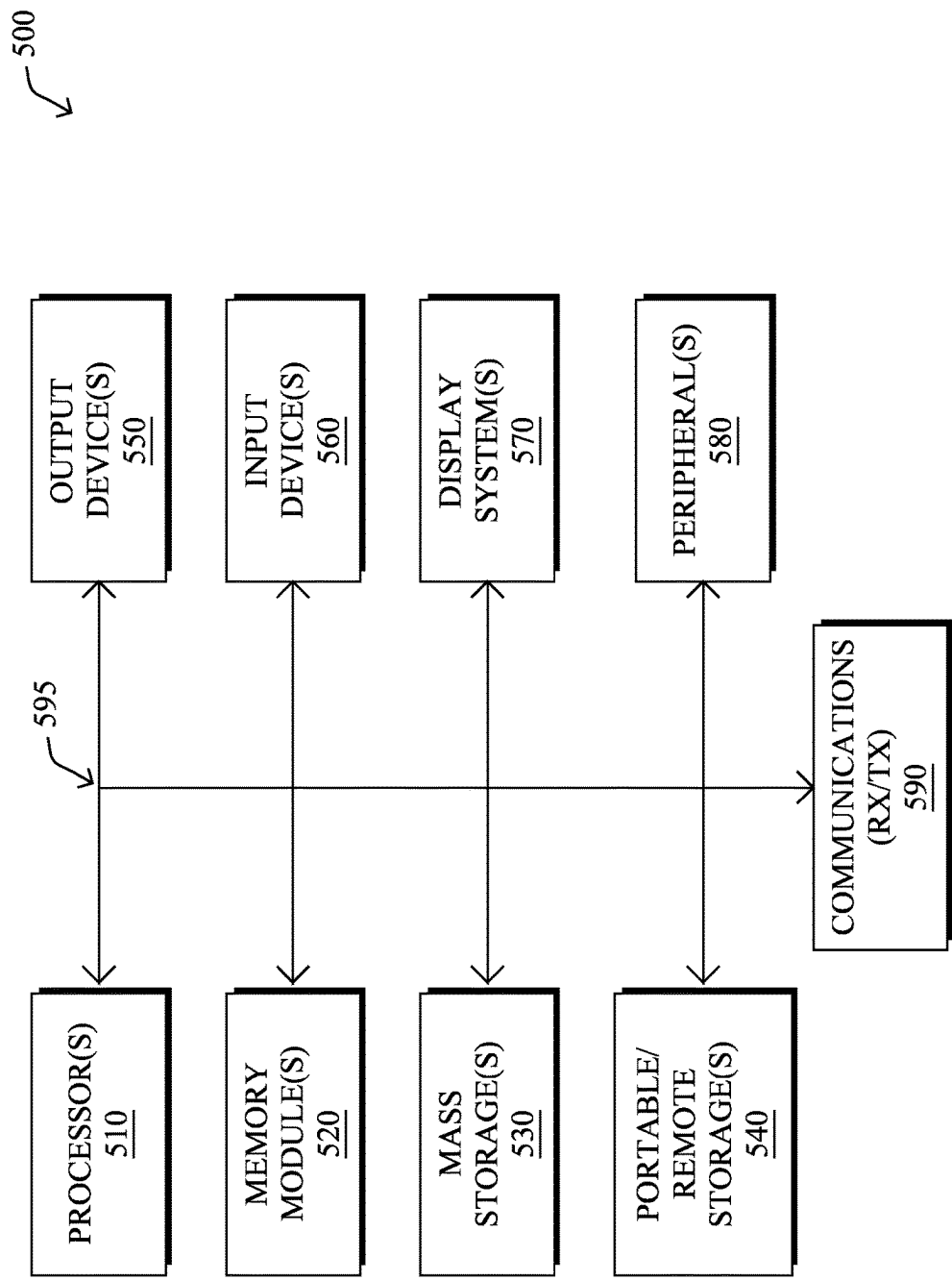


FIG. 5



600 ↘

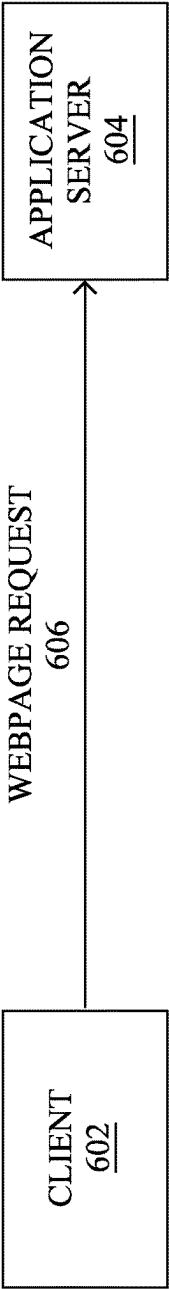


FIG. 6A

600 ↗

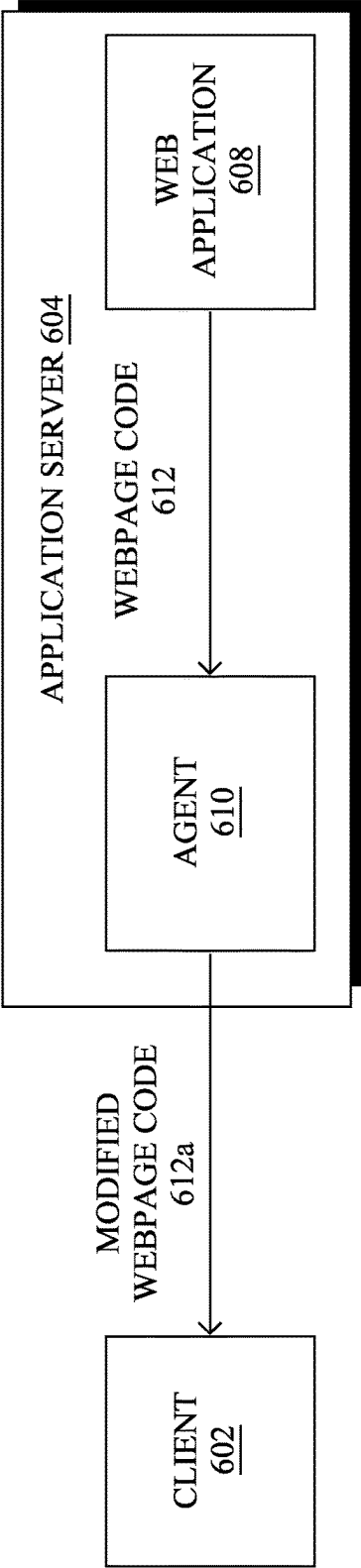


FIG. 6B

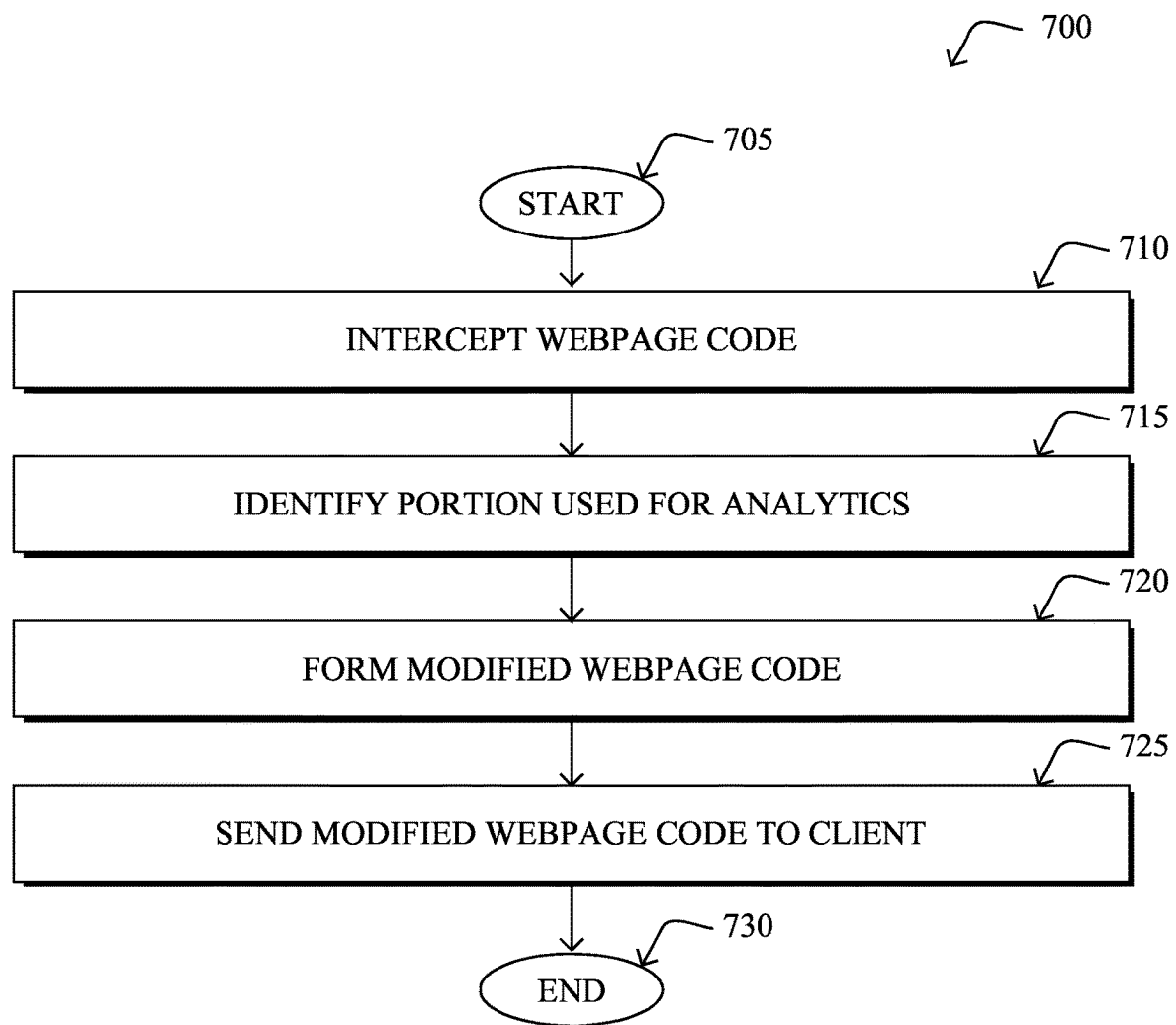


FIG. 7

# DYNAMIC WEB CONTENT MANAGEMENT SYSTEM

## TECHNICAL FIELD

The present disclosure relates generally to computer systems, and, more particularly, to a dynamic web content management system.

## BACKGROUND

The Internet and the World Wide Web have enabled the proliferation of web services available for virtually all types of businesses. Coupled with this proliferation has also been the rise of webpage analytics, which seek to capture metrics regarding website traffic. To do so, many webpage analytics tools operate by inserting web content such as tags, scripts, and the like into the code of a webpage under scrutiny. This causes a client accessing the webpage to send requests to the analytics service, which are then tracked by the analytics service for purposes of compiling reports.

While webpage analytics can offer useful information to a website operator, use of webpage analytics is also not without cost. Indeed, the additional requests sent by the client to the analytics service necessarily requires additional time and resources to perform, which could also increase the page load time of the webpage. To date, however, a “one size fits all” approach is usually taken with respect to webpage analytics whereby the analytics tags, scrips, etc. are manually inserted into the webpage code and always enabled.

## BRIEF DESCRIPTION OF THE DRAWINGS

The embodiments herein may be better understood by referring to the following description in conjunction with the accompanying drawings in which like reference numerals indicate identically or functionally similar elements, of which:

FIGS. 1A-1B illustrate an example computer network;  
FIG. 2 illustrates an example computing device/node;  
FIG. 3 illustrates an example application intelligence platform;

FIG. 4 illustrates an example system for implementing the example application intelligence platform;

FIG. 5 illustrates an example computing system implementing the disclosed technology;

FIGS. 6A-6B illustrate an example architecture for dynamic web content management; and

FIG. 7 illustrates an example simplified procedure for dynamic web content management.

## DESCRIPTION OF EXAMPLE EMBODIMENTS

### Overview

According to one or more embodiments of the disclosure, an agent executed by a device intercepts webpage code for a website sent from an application server to a client of the website. The agent identifies a portion of the webpage code as being used for webpage analytics. The agent forms modified webpage code by disabling the portion of the webpage code, based on one or more performance metrics associated with the website. The agent sends the modified webpage code to the client of the website.

Other embodiments are described below, and this overview is not meant to limit the scope of the present disclosure.

## DESCRIPTION

A computer network is a geographically distributed collection of nodes interconnected by communication links and segments for transporting data between end nodes, such as personal computers and workstations, or other devices, such as sensors, etc. Many types of networks are available, ranging from local area networks (LANs) to wide area networks (WANs). LANs typically connect the nodes over dedicated private communications links located in the same general physical location, such as a building or campus. WANs, on the other hand, typically connect geographically dispersed nodes over long-distance communications links, such as common carrier telephone lines, optical lightpaths, synchronous optical networks (SONET), synchronous digital hierarchy (SDH) links, or Powerline Communications (PLC), and others. The Internet is an example of a WAN that connects disparate networks throughout the world, providing global communication between nodes on various networks. Other types of networks, such as field area networks (FANs), neighborhood area networks (NANs), personal area networks (PANs), enterprise networks, etc. may also make up the components of any given computer network.

The nodes typically communicate over the network by exchanging discrete frames or packets of data according to predefined protocols, such as the Transmission Control Protocol/Internet Protocol (TCP/IP). In this context, a protocol consists of a set of rules defining how the nodes interact with each other. Computer networks may be further interconnected by an intermediate network node, such as a router, to extend the effective “size” of each network.

Smart object networks, such as sensor networks, in particular, are a specific type of network having spatially distributed autonomous devices such as sensors, actuators, etc., that cooperatively monitor physical or environmental conditions at different locations, such as, e.g., energy/power consumption, resource consumption (e.g., water/gas/etc. for advanced metering infrastructure or “AMI” applications) temperature, pressure, vibration, sound, radiation, motion, pollutants, etc. Other types of smart objects include actuators, e.g., responsible for turning on/off an engine or perform any other actions. Sensor networks, a type of smart object network, are typically shared-media networks, such as wireless or power-line communication networks. That is, in addition to one or more sensors, each sensor device (node) in a sensor network may generally be equipped with a radio transceiver or other communication port, a microcontroller, and an energy source, such as a battery. Generally, size and cost constraints on smart object nodes (e.g., sensors) result in corresponding constraints on resources such as energy, memory, computational speed and bandwidth.

FIG. 1A is a schematic block diagram of an example computer network 100 illustratively comprising nodes/devices, such as a plurality of routers/devices interconnected by links or networks, as shown. For example, customer edge (CE) routers 110 may be interconnected with provider edge (PE) routers 120 (e.g., PE-1, PE-2, and PE-3) in order to communicate across a core network, such as an illustrative network backbone 130. For example, routers 110, 120 may be interconnected by the public Internet, a multiprotocol label switching (MPLS) virtual private network (VPN), or the like. Data packets 140 (e.g., traffic/messages) may be exchanged among the nodes/devices of the computer network 100 over links using predefined network communication protocols such as the Transmission Control Protocol/Internet Protocol (TCP/IP), User Datagram Protocol (UDP), Asynchronous Transfer Mode (ATM) protocol, Frame Relay

protocol, or any other suitable protocol. Those skilled in the art will understand that any number of nodes, devices, links, etc. may be used in the computer network, and that the view shown herein is for simplicity.

In some implementations, a router or a set of routers may be connected to a private network (e.g., dedicated leased lines, an optical network, etc.) or a virtual private network (VPN), such as an MPLS VPN thanks to a carrier network, via one or more links exhibiting very different network and service level agreement characteristics.

FIG. 1B illustrates an example of network **100** in greater detail, according to various embodiments. As shown, network backbone **130** may provide connectivity between devices located in different geographical areas and/or different types of local networks. For example, network **100** may comprise local/branch networks **160**, **162** that include devices/nodes **10-16** and devices/nodes **18-20**, respectively, as well as a data center/cloud environment **150** that includes servers **152-154**. Notably, local networks **160-162** and data center/cloud environment **150** may be located in different geographic locations. Servers **152-154** may include, in various embodiments, any number of suitable servers or other cloud-based resources. As would be appreciated, network **100** may include any number of local networks, data centers, cloud environments, devices/nodes, servers, etc.

In some embodiments, the techniques herein may be applied to other network topologies and configurations. For example, the techniques herein may be applied to peering points with high-speed links, data centers, etc. Furthermore, in various embodiments, network **100** may include one or more mesh networks, such as an Internet of Things network. Loosely, the term “Internet of Things” or “IoT” refers to uniquely identifiable objects (things) and their virtual representations in a network-based architecture. In particular, the next frontier in the evolution of the Internet is the ability to connect more than just computers and communications devices, but rather the ability to connect “objects” in general, such as lights, appliances, vehicles, heating, ventilating, and air-conditioning (HVAC), windows and window shades and blinds, doors, locks, etc. The “Internet of Things” thus generally refers to the interconnection of objects (e.g., smart objects), such as sensors and actuators, over a computer network (e.g., via IP), which may be the public Internet or a private network.

Notably, shared-media mesh networks, such as wireless networks, are often on what is referred to as Low-Power and Lossy Networks (LLNs), which are a class of network in which both the routers and their interconnect are constrained: LLN routers typically operate with constraints, e.g., processing power, memory, and/or energy (battery), and their interconnects are characterized by, illustratively, high loss rates, low data rates, and/or instability. LLNs are comprised of anything from a few dozen to thousands or even millions of LLN routers, and support point-to-point traffic (between devices inside the LLN), point-to-multipoint traffic (from a central control point such as the root node to a subset of devices inside the LLN), and multipoint-to-point traffic (from devices inside the LLN towards a central control point). Often, an IoT network is implemented with an LLN-like architecture. For example, as shown, local network **160** may be an LLN in which CE-2 operates as a root node for nodes/devices **10-16** in the local mesh, in some embodiments.

FIG. 2 is a schematic block diagram of an example computing device (e.g., apparatus) **200** that may be used with one or more embodiments described herein, e.g., as any of the devices shown in FIGS. 1A-1B above, and particu-

larly as specific devices as described further below. The device may comprise one or more network interfaces **210** (e.g., wired, wireless, etc.), at least one processor **220**, and a memory **240** interconnected by a system bus **250**, as well as a power supply **260** (e.g., battery, plug-in, etc.).

The network interface(s) **210** contain the mechanical, electrical, and signaling circuitry for communicating data over links coupled to the network **100**, e.g., providing a data connection between device **200** and the data network, such as the Internet. The network interfaces may be configured to transmit and/or receive data using a variety of different communication protocols. For example, interfaces **210** may include wired transceivers, wireless transceivers, cellular transceivers, or the like, each to allow device **200** to communicate information to and from a remote computing device or server over an appropriate network. The same network interfaces **210** also allow communities of multiple devices **200** to interconnect among themselves, either peer-to-peer, or up and down a hierarchy. Note, further, that the nodes may have two different types of network connections via network interface(s) **210**, e.g., wireless and wired/physical connections, and that the view herein is merely for illustration. Also, while network interface(s) **210** are shown separately from power supply **260**, for devices using powerline communication (PLC) or Power over Ethernet (POE), the network interface **210** may communicate through the power supply **260**, or may be an integral component of the power supply.

The memory **240** comprises a plurality of storage locations that are addressable by the processor **220** and the network interfaces **210** for storing software programs and data structures associated with the embodiments described herein. The processor **220** may comprise hardware elements or hardware logic adapted to execute the software programs and manipulate the data structures **245**. An operating system **242**, portions of which are typically resident in memory **240** and executed by the processor, functionally organizes the device by, among other things, invoking operations in support of software processes and/or services executing on the device. These software processes and/or services may comprise one or more functional processes **246**, and on certain devices, an illustrative monitoring process **248**, as described herein. Notably, functional processes **246**, when executed by processor(s) **220**, cause each particular device **200** to perform the various functions corresponding to the particular device's purpose and general configuration. For example, a router would be configured to operate as a router, a server would be configured to operate as a server, an access point (or gateway) would be configured to operate as an access point (or gateway), a client device would be configured to operate as a client device, and so on.

It will be apparent to those skilled in the art that other processor and memory types, including various computer-readable media, may be used to store and execute program instructions pertaining to the techniques described herein. Also, while the description illustrates various processes, it is expressly contemplated that various processes may be embodied as modules configured to operate in accordance with the techniques herein (e.g., according to the functionality of a similar process). Further, while the processes have been shown separately, those skilled in the art will appreciate that processes may be routines or modules within other processes.

#### Application Intelligence Platform

The embodiments herein relate to an application intelligence platform for application performance management. In

## 5

one aspect, as discussed with respect to FIGS. 3-5 below, performance within a networking environment may be monitored, specifically by monitoring applications and entities (e.g., transactions, tiers, nodes, and machines) in the networking environment using agents installed at individual machines at the entities. As an example, applications may be configured to run on one or more machines (e.g., a customer will typically run one or more nodes on a machine, where an application consists of one or more tiers, and a tier consists of one or more nodes). The agents collect data associated with the applications of interest and associated nodes and machines where the applications are being operated. Examples of the collected data may include performance data (e.g., metrics, metadata, etc.) and topology data (e.g., indicating relationship information). The agent-collected data may then be provided to one or more servers or controllers to analyze the data.

FIG. 3 is a block diagram of an example application intelligence platform 300 that can implement one or more aspects of the techniques herein. The application intelligence platform is a system that monitors and collects metrics of performance data for an application environment being monitored. At the simplest structure, the application intelligence platform includes one or more agents 310 and one or more servers/controllers 320. Note that while FIG. 3 shows four agents (e.g., Agent 1 through Agent 4) communicatively linked to a single controller, the total number of agents and controllers can vary based on a number of factors including the number of applications monitored, how distributed the application environment is, the level of monitoring desired, the level of user experience desired, and so on.

The controller 320 is the central processing and administration server for the application intelligence platform. The controller 320 serves a browser-based user interface (UI) 330 that is the primary interface for monitoring, analyzing, and troubleshooting the monitored environment. The controller 320 can control and manage monitoring of business transactions (described below) distributed over application servers. Specifically, the controller 320 can receive runtime data from agents 310 (and/or other coordinator devices), associate portions of business transaction data, communicate with agents to configure collection of runtime data, and provide performance data and reporting through the interface 330. The interface 330 may be viewed as a web-based interface viewable by a client device 340. In some implementations, a client device 340 can directly communicate with controller 320 to view an interface for monitoring data. The controller 320 can include a visualization system 350 for displaying the reports and dashboards related to the disclosed technology. In some implementations, the visualization system 350 can be implemented in a separate machine (e.g., a server) different from the one hosting the controller 320.

Notably, in an illustrative Software as a Service (SaaS) implementation, a controller instance may be hosted remotely by a provider of the application intelligence platform 300. In an illustrative on-premises (On-Prem) implementation, a controller instance may be installed locally and self-administered.

The controllers 320 receive data from different agents 310 (e.g., Agents 1-4) deployed to monitor applications, databases and database servers, servers, and end user clients for the monitored environment. Any of the agents 310 can be implemented as different types of agents with specific monitoring duties. For example, application agents may be installed on each server that hosts applications to be moni-

## 6

tored. Instrumenting an agent adds an application agent into the runtime process of the application.

Database agents, for example, may be software (e.g., a Java™ program) installed on a machine that has network access to the monitored databases and the controller. Database agents query the monitored databases in order to collect metrics and pass those metrics along for display in a metric browser (e.g., for database monitoring and analysis within databases pages of the controller's UI 330). Multiple database agents can report to the same controller. Additional database agents can be implemented as backup database agents to take over for the primary database agents during a failure or planned machine downtime. The additional database agents can run on the same machine as the primary agents or on different machines. A database agent can be deployed in each distinct network of the monitored environment. Multiple database agents can run under different user accounts on the same machine.

Standalone machine agents, on the other hand, may be standalone programs (e.g., standalone Java™ programs) that collect hardware-related performance statistics from the servers (or other suitable devices) in the monitored environment. The standalone machine agents can be deployed on machines that host application servers, database servers, messaging servers, Web servers, etc. A standalone machine agent has an extensible architecture (e.g., designed to accommodate changes).

End user monitoring (EUM) may be performed using browser agents and mobile agents to provide performance information from the point of view of the client, such as a web browser or a mobile native application. Through EUM, web use, mobile use, or combinations thereof (e.g., by real users or synthetic agents) can be monitored based on the monitoring needs. Notably, browser agents (e.g., agents 310) can include Reporters that report monitored data to the controller.

Monitoring through browser agents and mobile agents are generally unlike monitoring through application agents, database agents, and standalone machine agents that are on the server. In particular, browser agents may generally be embodied as small files using web-based technologies, such as JavaScript™ agents injected into each instrumented web page (e.g., as close to the top as possible) as the web page is served, and are configured to collect data. Once the web page has completed loading, the collected data may be bundled into a beacon and sent to an EUM process/cloud for processing and made ready for retrieval by the controller. Browser real user monitoring (Browser RUM) provides insights into the performance of a web application from the point of view of a real or synthetic end user. For example, Browser RUM can determine how specific Ajax or iframe calls are slowing down page load time and how server performance impact end user experience in aggregate or in individual cases.

A mobile agent, on the other hand, may be a small piece of highly performant code that gets added to the source of the mobile application. Mobile RUM provides information on the native mobile application (e.g., iOS or Android applications) as the end users actually use the mobile application. Mobile RUM provides visibility into the functioning of the mobile application itself and the mobile application's interaction with the network used and any server-side applications with which the mobile application communicates.

Application Intelligence Monitoring: The disclosed technology can provide application intelligence data by monitoring an application environment that includes various

services such as web applications served from an application server (e.g., Java™ virtual machine (JVM), Internet Information Services (IIS), Hypertext Preprocessor (PHP) Web server, etc.), databases or other data stores, and remote services such as message queues and caches. The services in the application environment can interact in various ways to provide a set of cohesive user interactions with the application, such as a set of user services applicable to end user customers.

Application Intelligence Modeling: Entities in the application environment (such as the JBoss service, MQSeries modules, and databases) and the services provided by the entities (such as a login transaction, service or product search, or purchase transaction) may be mapped to an application intelligence model. In the application intelligence model, a business transaction represents a particular service provided by the monitored environment. For example, in an e-commerce application, particular real-world services can include a user logging in, searching for items, or adding items to the cart. In a content portal, particular real-world services can include user requests for content such as sports, business, or entertainment news. In a stock trading application, particular real-world services can include operations such as receiving a stock quote, buying, or selling stocks.

Business Transactions: A business transaction representation of the particular service provided by the monitored environment provides a view on performance data in the context of the various tiers that participate in processing a particular request. A business transaction, which may each be identified by a unique business transaction identification (ID), represents the end-to-end processing path used to fulfill a service request in the monitored environment (e.g., adding items to a shopping cart, storing information in a database, purchasing an item online, etc.). Thus, a business transaction is a type of user-initiated action in the monitored environment defined by an entry point and a processing path across application servers, databases, and potentially many other infrastructure components. Each instance of a business transaction is an execution of that transaction in response to a particular user request (e.g., a socket call, illustratively associated with the TCP layer). A business transaction can be created by detecting incoming requests at an entry point and tracking the activity associated with request at the originating tier and across distributed components in the application environment (e.g., associating the business transaction with a 4-tuple of a source IP address, source port, destination IP address, and destination port). A flow map can be generated for a business transaction that shows the touch points for the business transaction in the application environment. In one embodiment, a specific tag may be added to packets by application specific agents for identifying business transactions (e.g., a custom header field attached to a HyperText Transfer Protocol (HTTP) payload by an application agent, or by a network agent when an application makes a remote socket call), such that packets can be examined by network agents to identify the business transaction identifier (ID) (e.g., a Globally Unique Identifier (GUID) or Universally Unique Identifier (UUID)).

Performance monitoring can be oriented by business transaction to focus on the performance of the services in the application environment from the perspective of end users. Performance monitoring based on business transactions can provide information on whether a service is available (e.g., users can log in, check out, or view their data), response times for users, and the cause of problems when the problems occur.

A business application is the top-level container in the application intelligence model. A business application contains a set of related services and business transactions. In some implementations, a single business application may be needed to model the environment. In some implementations, the application intelligence model of the application environment can be divided into several business applications. Business applications can be organized differently based on the specifics of the application environment. One consideration is to organize the business applications in a way that reflects work teams in a particular organization, since role-based access controls in the Controller UI are oriented by business application.

A node in the application intelligence model corresponds to a monitored server or JVM in the application environment. A node is the smallest unit of the modeled environment. In general, a node corresponds to an individual application server, JVM, or Common Language Runtime (CLR) on which a monitoring Agent is installed. Each node identifies itself in the application intelligence model. The Agent installed at the node is configured to specify the name of the node, tier, and business application under which the Agent reports data to the Controller.

Business applications contain tiers, the unit in the application intelligence model that includes one or more nodes. Each node represents an instrumented service (such as a web application). While a node can be a distinct application in the application environment, in the application intelligence model, a node is a member of a tier, which, along with possibly many other tiers, make up the overall logical business application.

Tiers can be organized in the application intelligence model depending on a mental model of the monitored application environment. For example, identical nodes can be grouped into a single tier (such as a cluster of redundant servers). In some implementations, any set of nodes, identical or not, can be grouped for the purpose of treating certain performance metrics as a unit into a single tier.

The traffic in a business application flows among tiers and can be visualized in a flow map using lines among tiers. In addition, the lines indicating the traffic flows among tiers can be annotated with performance metrics. In the application intelligence model, there may not be any interaction among nodes within a single tier. Also, in some implementations, an application agent node cannot belong to more than one tier. Similarly, a machine agent cannot belong to more than one tier. However, more than one machine agent can be installed on a machine.

A backend is a component that participates in the processing of a business transaction instance. A backend is not instrumented by an agent. A backend may be a web server, database, message queue, or other type of service. The agent recognizes calls to these backend services from instrumented code (called exit calls). When a service is not instrumented and cannot continue the transaction context of the call, the agent determines that the service is a backend component. The agent picks up the transaction context at the response at the backend and continues to follow the context of the transaction from there.

Performance information is available for the backend call. For detailed transaction analysis for the leg of a transaction processed by the backend, the database, web service, or other application need to be instrumented.

The application intelligence platform uses both self-learned baselines and configurable thresholds to help identify application issues. A complex distributed application has a large number of performance metrics and each metric is

important in one or more contexts. In such environments, it is difficult to determine the values or ranges that are normal for a particular metric; set meaningful thresholds on which to base and receive relevant alerts; and determine what is a “normal” metric when the application or infrastructure undergoes change. For these reasons, the disclosed application intelligence platform can perform anomaly detection based on dynamic baselines or thresholds.

The disclosed application intelligence platform automatically calculates dynamic baselines for the monitored metrics, defining what is “normal” for each metric based on actual usage. The application intelligence platform uses these baselines to identify subsequent metrics whose values fall out of this normal range. Static thresholds that are tedious to set up and, in rapidly changing application environments, error-prone, are no longer needed.

The disclosed application intelligence platform can use configurable thresholds to maintain service level agreements (SLAs) and ensure optimum performance levels for system by detecting slow, very slow, and stalled transactions. Configurable thresholds provide a flexible way to associate the right business context with a slow request to isolate the root cause.

In addition, health rules can be set up with conditions that use the dynamically generated baselines to trigger alerts or initiate other types of remedial actions when performance problems are occurring or may be about to occur.

For example, dynamic baselines can be used to automatically establish what is considered normal behavior for a particular application. Policies and health rules can be used against baselines or other health indicators for a particular application to detect and troubleshoot problems before users are affected. Health rules can be used to define metric conditions to monitor, such as when the “average response time is four times slower than the baseline”. The health rules can be created and modified based on the monitored application environment.

Examples of health rules for testing business transaction performance can include business transaction response time and business transaction error rate. For example, health rule that tests whether the business transaction response time is much higher than normal can define a critical condition as the combination of an average response time greater than the default baseline by 3 standard deviations and a load greater than 50 calls per minute. In some implementations, this health rule can define a warning condition as the combination of an average response time greater than the default baseline by 2 standard deviations and a load greater than 100 calls per minute. In some implementations, the health rule that tests whether the business transaction error rate is much higher than normal can define a critical condition as the combination of an error rate greater than the default baseline by 3 standard deviations and an error rate greater than 10 errors per minute and a load greater than 50 calls per minute. In some implementations, this health rule can define a warning condition as the combination of an error rate greater than the default baseline by 2 standard deviations and an error rate greater than 5 errors per minute and a load greater than 50 calls per minute. These are non-exhaustive and non-limiting examples of health rules and other health rules can be defined as desired by the user.

Policies can be configured to trigger actions when a health rule is violated or when any event occurs. Triggered actions can include notifications, diagnostic actions, auto-scaling capacity, running remediation scripts.

Most of the metrics relate to the overall performance of the application or business transaction (e.g., load, average

response time, error rate, etc.) or of the application server infrastructure (e.g., percentage CPU busy, percentage of memory used, etc.). The Metric Browser in the controller UI can be used to view all of the metrics that the agents report to the controller.

In addition, special metrics called information points can be created to report on how a given business (as opposed to a given application) is performing. For example, the performance of the total revenue for a certain product or set of products can be monitored. Also, information points can be used to report on how a given code is performing, for example how many times a specific method is called and how long it is taking to execute. Moreover, extensions that use the machine agent can be created to report user defined custom metrics. These custom metrics are base-lined and reported in the controller, just like the built-in metrics.

All metrics can be accessed programmatically using a Representational State Transfer (REST) API that returns either the JavaScript™ Object Notation (JSON) or the extensible Markup Language (XML) format. Also, the REST API can be used to query and manipulate the application environment.

Snapshots provide a detailed picture of a given application at a certain point in time. Snapshots usually include call graphs that allow that enables drilling down to the line of code that may be causing performance problems. The most common snapshots are transaction snapshots.

FIG. 4 illustrates an example application intelligence platform (system) 400 for performing one or more aspects of the techniques herein. The system 400 in FIG. 4 includes client 405, client device 492, mobile device 415, network 420, network server 425, application servers 430, 440, 450, and 460, asynchronous network machine 470, data stores 480 and 485, controller 490, and data collection server 495. The controller 490 can include visualization system 496 for providing displaying of the report generated for performing the field name recommendations for field extraction as disclosed in the present disclosure. In some implementations, the visualization system 496 can be implemented in a separate machine (e.g., a server) different from the one hosting the controller 490.

Client 405 may include network browser 410 and be implemented as a computing device, such as for example a laptop, desktop, workstation, or some other computing device. Network browser 410 may be a client application for viewing content provided by an application server, such as application server 430 via network server 425 over network 420.

Network browser 410 may include agent 412. Agent 412 may be installed on network browser 410 and/or client 405 as a network browser add-on, downloading the application to the server, or in some other manner. Agent 412 may be executed to monitor network browser 410, the operating system of client 405, and any other application, API, or another component of client 405. Agent 412 may determine network browser navigation timing metrics, access browser cookies, monitor code, and transmit data to data collection server 495, controller 490, or another device. Agent 412 may perform other operations related to monitoring a request or a network at client 405 as discussed herein including report generating.

Mobile device 415 is connected to network 420 and may be implemented as a portable device suitable for sending and receiving content over a network, such as for example a mobile phone, smart phone, tablet computer, or other portable device. Both client 405 and mobile device 415 may



include hardware and/or software configured to access a web service provided by network server **425**.

Mobile device **415** may include network browser **417** and an agent **419**. Mobile device may also include client applications and other code that may be monitored by agent **419**. Agent **419** may reside in and/or communicate with network browser **417**, as well as communicate with other applications, an operating system, APIs and other hardware and software on mobile device **415**. Agent **419** may have similar functionality as that described herein for agent **412** on client **405**, and may report data to data collection server **495** and/or controller **490**.

Network **420** may facilitate communication of data among different servers, devices and machines of system **400** (some connections shown with lines to network **420**, some not shown). The network may be implemented as a private network, public network, intranet, the Internet, a cellular network, Wi-Fi network, VoIP network, or a combination of one or more of these networks. The network **420** may include one or more machines such as load balance machines and other machines.

Network server **425** is connected to network **420** and may receive and process requests received over network **420**. Network server **425** may be implemented as one or more servers implementing a network service, and may be implemented on the same machine as application server **430** or one or more separate machines. When network **420** is the Internet, network server **425** may be implemented as a web server.

Application server **430** communicates with network server **425**, application servers **440** and **450**, and controller **490**. Application server **450** may also communicate with other machines and devices (not illustrated in FIG. 4). Application server **430** may host an application or portions of a distributed application. The host application **432** may be in one of many platforms, such as including a Java™, PHP, .Net, and Node.JS, be implemented as a Java™ virtual machine, or include some other host type. Application server **430** may also include one or more agents **434** (i.e., “modules”), including a language agent, machine agent, and network agent, and other software modules. Application server **430** may be implemented as one server or multiple servers as illustrated in FIG. 4.

Application **432** and other software on application server **430** may be instrumented using byte code insertion, or byte code instrumentation (BCI), to modify the object code of the application or other software. The instrumented object code may include code used to detect calls received by application **432**, calls sent by application **432**, and communicate with agent **434** during execution of the application. BCI may also be used to monitor one or more sockets of the application and/or application server in order to monitor the socket and capture packets coming over the socket.

In some embodiments, server **430** may include applications and/or code other than a virtual machine. For example, servers **430**, **440**, **450**, and **460** may each include Java™ code, .Net code, PHP code, Ruby code, C code, C++ or other binary code to implement applications and process requests received from a remote source. References to a virtual machine with respect to an application server are intended to be for exemplary purposes only.

Agents **434** on application server **430** may be installed, downloaded, embedded, or otherwise provided on application server **430**. For example, agents **434** may be provided in server **430** by instrumentation of object code, downloading the agents to the server, or in some other manner. Agent **434** may be executed to monitor application server **430**,

monitor application **432** running in a virtual machine (or other program language, such as a PHP, .Net, or C program), machine resources, network layer data, and communicate with byte instrumented code on application server **430** and one or more applications on application server **430**.

Each of agents **434**, **444**, **454**, and **464** may include one or more agents, such as language agents, machine agents, and network agents. A language agent may be a type of agent that is suitable to run on a particular host. Examples of language agents include a Java™ agent, .Net agent, PHP agent, and other agents. The machine agent may collect data from a particular machine on which it is installed. A network agent may capture network information, such as data collected from a socket.

Agent **434** may detect operations such as receiving calls and sending requests by application server **430**, resource usage, and incoming packets. Agent **434** may receive data, process the data, for example by aggregating data into metrics, and transmit the data and/or metrics to controller **490**. Agent **434** may perform other operations related to monitoring applications and application server **430** as discussed herein. For example, agent **434** may identify other applications, share business transaction data, aggregate detected runtime data, and other operations.

An agent may operate to monitor a node, tier of nodes, or other entity. A node may be a software program or a hardware component (e.g., memory, processor, and so on). A tier of nodes may include a plurality of nodes which may process a similar business transaction, may be located on the same server, may be associated with each other in some other way, or may not be associated with each other.

A language agent may be an agent suitable to instrument or modify, collect data from, and reside on a host. The host may be a Java, PHP, .Net, Node.JS, or other type of platform. Language agents may collect flow data as well as data associated with the execution of a particular application. The language agent may instrument the lowest level of the application to gather the flow data. The flow data may indicate which tier is communicating with which tier and on which port. In some instances, the flow data collected from the language agent includes a source IP, a source port, a destination IP, and a destination port. The language agent may report the application data and call chain data to a controller. The language agent may report the collected flow data associated with a particular application to a network agent.

A network agent may be a standalone agent that resides on the host and collects network flow group data. The network flow group data may include a source IP, destination port, destination IP, and protocol information for network flow received by an application on which network agent is installed. The network agent may collect data by intercepting and performing packet capture on packets coming in from one or more network interfaces (e.g., so that data generated/received by all the applications using sockets can be intercepted). The network agent may receive flow data from a language agent that is associated with applications to be monitored. For flows in the flow group data that match flow data provided by the language agent, the network agent rolls up the flow data to determine metrics such as TCP throughput, TCP loss, latency, and bandwidth. The network agent may then report the metrics, flow group data, and call chain data to a controller. The network agent may also make system calls at an application server to determine system information, such as for example a host status check, a network status check, socket status, and other information.

A machine agent, which may be referred to as an infrastructure agent, may reside on the host and collect information regarding the machine which implements the host. A machine agent may collect and generate metrics from information such as processor usage, memory usage, and other hardware information.

Each of the language agent, network agent, and machine agent may report data to the controller. Controller 490 may be implemented as a remote server that communicates with agents located on one or more servers or machines. The controller may receive metrics, call chain data and other data, correlate the received data as part of a distributed transaction, and report the correlated data in the context of a distributed application implemented by one or more monitored applications and occurring over one or more monitored networks. The controller may provide reports, one or more user interfaces, and other information for a user.

Agent 434 may create a request identifier for a request received by server 430 (for example, a request received by a client 405 or mobile device 415 associated with a user or another source). The request identifier may be sent to client 405 or mobile device 415, whichever device sent the request. In embodiments, the request identifier may be created when data is collected and analyzed for a particular business transaction.

Each of application servers 440, 450, and 460 may include an application and agents. Each application may run on the corresponding application server. Each of applications 442, 452, and 462 on application servers 440-460 may operate similarly to application 432 and perform at least a portion of a distributed business transaction. Agents 444, 454, and 464 may monitor applications 442-462, collect and process data at runtime, and communicate with controller 490. The applications 432, 442, 452, and 462 may communicate with each other as part of performing a distributed transaction. Each application may call any application or method of another virtual machine.

Asynchronous network machine 470 may engage in asynchronous communications with one or more application servers, such as application server 450 and 460. For example, application server 450 may transmit several calls or messages to an asynchronous network machine. Rather than communicate back to application server 450, the asynchronous network machine may process the messages and eventually provide a response, such as a processed message, to application server 460. Because there is no return message from the asynchronous network machine to application server 450, the communications among them are asynchronous.

Data stores 480 and 485 may each be accessed by application servers such as application server 460. Data store 485 may also be accessed by application server 450. Each of data stores 480 and 485 may store data, process data, and return queries received from an application server. Each of data stores 480 and 485 may or may not include an agent.

Controller 490 may control and manage monitoring of business transactions distributed over application servers 430-460. In some embodiments, controller 490 may receive application data, including data associated with monitoring client requests at client 405 and mobile device 415, from data collection server 495. In some embodiments, controller 490 may receive application monitoring data and network data from each of agents 412, 419, 434, 444, and 454 (also referred to herein as "application monitoring agents"). Controller 490 may associate portions of business transaction data, communicate with agents to configure collection of data, and provide performance data and reporting through an

interface. The interface may be viewed as a web-based interface viewable by client device 492, which may be a mobile device, client device, or any other platform for viewing an interface provided by controller 490. In some embodiments, a client device 492 may directly communicate with controller 490 to view an interface for monitoring data.

Client device 492 may include any computing device, including a mobile device or a client computer such as a desktop, work station or other computing device. Client device 492 may communicate with controller 490 to create and view a custom interface. In some embodiments, controller 490 provides an interface for creating and viewing the custom interface as a content page, e.g., a web page, which may be provided to and rendered through a network browser application on client device 492.

Applications 432, 442, 452, and 462 may be any of several types of applications. Examples of applications that may implement applications 432-462 include a Java, PHP, .Net, Node.JS, and other applications.

FIG. 5 is a block diagram of a computer system 500 for implementing the present technology, which is a specific implementation of device 200 of FIG. 2 above. System 500 of FIG. 5 may be implemented in the contexts of the likes of client 405, client device 492, network server 425, servers 430, 440, 450, 460, asynchronous network machine 470, and controller 490 of FIG. 4. (Note that the specifically configured system 500 of FIG. 5 and the customized device 200 of FIG. 2 are not meant to be mutually exclusive, and the techniques herein may be performed by any suitably configured computing device.)

The computing system 500 of FIG. 5 includes one or more processor(s) 510 and memory 520. Main memory 520 stores, in part, instructions and data for execution by processor(s) 510. Main memory 520 can store the executable code when in operation. The system 500 of FIG. 5 further includes a mass storage device 530, portable/remote storage(s) 540, output devices 550, user input devices 560, display system(s) 570, and peripheral(s) 580.

The components shown in FIG. 5 are depicted as being connected via a single bus 590. However, the components may be connected through one or more data transport means. For example, processor(s) 510 and main memory 520 may be connected via a local microprocessor bus, and the mass storage device 530, peripheral(s) 580, storage(s) 540, and display system(s) 570 may be connected via one or more input/output (I/O) buses.

Mass storage device 530, which may be implemented with a magnetic disk drive or an optical disk drive, is a non-volatile storage device for storing data and instructions for use by processor(s) 510. Mass storage device 530 can store the system software for implementing embodiments of the present disclosure for purposes of loading that software into main memory 520.

Portable/remote storage(s) 540 may operate in conjunction with a portable non-volatile storage medium, such as a compact disk, digital video disk, magnetic disk, flash storage, etc. to input and output data and code to and from the computer system 500 of FIG. 5. The system software for implementing embodiments of the present disclosure may be stored on such a portable medium and input to the computer system 500 via the storage(s) 540.

Input devices 560 provide a portion of a user interface. Input devices 560 may include an alpha-numeric keypad, such as a keyboard, for inputting alpha-numeric and other information, or a pointing device, such as a mouse, a trackball, stylus, or cursor direction keys. Additionally, the system 500 as shown in FIG. 5 includes output devices 550.

15

Examples of suitable output devices include speakers, printers, network interfaces, and monitors.

Display system(s) 570 may include a liquid crystal display (LCD) or other suitable display device. Display system(s) 570 receives textual and graphical information, and processes the information for output to the display device.

Peripheral(s) 580 may include any type of computer support device to add additional functionality to the computer system. For example, peripheral(s) 580 may include a modem or a router.

The components contained in the computer system 500 of FIG. 5 can include a personal computer, handheld computing device, telephone, mobile computing device, workstation, server, minicomputer, mainframe computer, or any other computing device. The computer can also include different bus configurations, networked platforms, multi-processor platforms, etc. Various operating systems can be used including Unix™, Linux™, Windows™, Apple™ OS, and other suitable operating systems, including mobile versions.

When implementing a mobile device such as smart phone or tablet computer, the computer system 500 of FIG. 5 may include one or more antennas, radios, and other circuitry for communicating over wireless signals, such as for example communication using Wi-Fi, cellular, or other wireless signals.

As noted above, many websites utilize webpage analytics tools, to capture metrics regarding their user traffic. To do so, web content such as tags, scripts, and the like may be inserted into the webpage code. This causes a client accessing the webpage to send requests to the analytics service, which are then tracked by the analytics service for purposes of compiling reports.

While webpage analytics can offer useful information to a website operator, use of webpage analytics is also not without cost. Indeed, the additional requests sent by the client to the analytics service necessarily requires additional time and resources to perform, which could also increase the page load time of the webpage. To date, however, a “one size fits all” approach is usually taken with respect to webpage analytics whereby the analytics tags, scrips, etc. are manually inserted into the webpage code and always enabled.

#### Dynamic Web Content Management System

The techniques introduced herein allow for the dynamic management of web content, such as those inserted into webpage code for purposes of analytics. In some aspects, an agent may intercept webpage code being sent to a client and dynamically decide to disable the tags, scripts, or the like, inserted into the webpage code, based on any number of factors. For instance, in some embodiments, the web content for the webpage analytics may be disabled dynamically based on factors such as performance metrics associated with the website (e.g., page load times, traffic loads, etc.), the locations of the clients, security risk factors, combinations thereof, or the like.

Illustratively, the techniques described herein may be performed by hardware, software, and/or firmware, such as in accordance with the illustrative application monitoring process 248, which may include computer executable instructions executed by the processor 220 to perform functions relating to the techniques described herein, e.g., in conjunction with corresponding processes of other devices in the computer network as described herein (e.g., on network agents, controllers, computing devices, servers, etc.).

16

Specifically, according to various embodiments, an agent executed by a device intercepts webpage code for a website sent from an application server to a client of the website. The agent identifies a portion of the webpage code as being used for webpage analytics. The agent forms modified webpage code by disabling the portion of the webpage code, based on one or more performance metrics associated with the website. The agent sends the modified webpage code to the client of the website.

Operationally, FIGS. 6A-6B illustrate an example architecture 600 for dynamic web content management, according to various embodiments. As shown in FIG. 6A, consider the case in which a client 602 (e.g., a client 405, a mobile device 415, etc.) sends a webpage request 606 to an application server 604 configured as a webserver for a website. For instance, webpage request 606 may be sent to a specific uniform resource indicator (URI) or uniform resource locator (URL) that resolves to a certain IP address associated with application server 604, requesting access to a specific webpage of the website.

As would be appreciated, various exchanges may be performed between client 602 and application server, in conjunction with webpage request 606. For instance, if HTTP Secure (HTTPS) is used, client 602 and application server 604 may also perform a cryptographic exchange, such as in accordance with the Transport Layer Security (TLS) protocol or the like, to secure the communications sent between client 602 and application server 604.

In addition, in some embodiments, an authentication exchange may also take place between client 602 and application server 604. For instance, the webpage being requested by webpage request 606 may be protected and only accessible to authorized users. In such cases, the user of client 602 may also log into their account with the website served by application server 604. This login process may entail simply providing a recognized username and password to application server 604. In more complex scenarios, the login process may entail multifactor authentication (MFA) or continuous MFA (cMFA), such as by requiring the user of client 602 to provide biometric data, a verification code sent via text, phone, email or the like, and/or other information that can be used to prove the identity of the user to application server 604.

According to various embodiments, as shown in FIG. 6B, the techniques herein propose the use of an agent 610 (e.g., any of agents 310, 434, 444, 454, 464, etc.) that is configured to intercept the webpage code 612 returned by the web application 608 of application server 604 in response to a webpage request. For instance, as shown, web application 608 may send webpage code 612 towards client 602, in response to receiving the webpage request 606 sent by client 602 in FIG. 6A.

In various embodiments, agent 610 may be executed directly on application server 604, thereby performing the interception of agent 610, locally. However, in further embodiments, agent 610 may also be implemented as a cloud access security broker (CASB), as a sidcar proxy (e.g., a Kubernetes sidcar/Envoy proxy), on an intermediate device between application server 604 and client 602, or the like.

A key function of agent 610 is to modify webpage code 612, as needed and in accordance with a defined policy, in various embodiments. In turn, agent 610 may send the modified webpage code 612a to the client that sent the webpage request, such as client 602. Such a policy may be set by default and/or specified via a user interface with agent 610, configuration file, or the like. Example factors consid-

ered by the policy for purposes of determining whether to modify webpage code **612** may include, but are not limited to, any or all of the following:

One or more performance metrics associated with the website—for instance, such metrics may include page load times, traffic load metrics, or the like.

Whether the webpage request was sent by a bot—for instance, agent **610** may opt to disable a portion of webpage code **612** when client **602** is a bot or vice-versa.

Whether the requesting client was logged into an account associated with the website

The identity of the user—for instance, agent **610** may opt to disable a portion of webpage code **612** for a certain subset of users.

The location of the client—for instance, agent **610** may estimate the geolocation of client **602** based on its IP address or other information and use this information to determine whether to modify webpage code **612**.

for non-linear models), replicating reservoir networks (e.g., for non-linear models, typically for timeseries), random forest classification, or the like

According to various embodiments, agent **610** may modify webpage code **612** to form modified webpage code **612a** by disabling at least a portion webpage code **612**, such as a portion that is used for website analytics. Such detection and modification may depend on the type of content included in webpage code **612** for purposes of the analytics. For instance, agent **610** may assess webpage code **612** to identify web metadata tags, script tags, or the like.

In the case of web page metadata tags, webpage code **612** may include a HyperText Markup Language (HTML) meta tag elements of the form “<meta>,” which are often included in the <head> element of the webpage code. Generally, metadata is the data that describes the underlying data and the <meta> element in HTML can be used to signify such information. For instance, webpage code **612** may include a metadata tag similar to the following:

---

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <meta name="Description" CONTENT="Author: A.N. Author, Illustrator: P.
Picture, Category: Books, Price: ??9.24, Length: 784 pages">
    <meta name="google-site-verification"
content="+nxGUDJ4QpAZ519Bsidi102tLVC21Alh5d1N123908vVuFHs34="/>
    <title>Example Books - high-quality used books for children</title>
    <meta name="robots" content="noindex,nofollow">
  </head>
</html>
```

---

Whether the content poses a security risk—for instance, agent **610** may assess webpage code **612** to determine whether the content poses a risk to disclosing sensitive data, presents a potential vulnerability, or the like.

Etc.

In some embodiments, agent **610** may leverage machine learning, to determine whether to modify webpage code **612** or not, based on any or all of the above factors (e.g., by estimating or forecasting certain information). Example machine learning techniques that agent **610** can employ may include, but are not limited to, nearest neighbor (NN) techniques (e.g., k-NN models, replicator NN models, etc.), statistical techniques (e.g., Bayesian networks, etc.), clustering techniques (e.g., k-means, mean-shift, etc.), neural networks (e.g., reservoir networks, artificial neural networks, etc.), support vector machines (SVMs), generative adversarial networks (GANs), long short-term memory (LSTM), logistic or other regression, Markov models or chains, principal component analysis (PCA) (e.g., for linear models), singular value decomposition (SVD), multi-layer perceptron (MLP) artificial neural networks (ANNs) (e.g.,

Such metadata tags are often used by search engines and can be abused by malicious entities to trick users into going to sites that mimic that of a legitimate website. In addition, these types of tags can also expose sensitive information to bots, search engines, and even to hackers. Thus, the use of metadata tags in webpage code **612** may be audited, regulated, and/or selectively allowed by agent **610**. In such cases, agent **610** may modify webpage code **612** to form modified webpage code **612a** for delivery to client **602** by removing any portions thereof that include metadata tags (e.g., those identified by agent **610** that are used for analytics or other purposes).

In further embodiments, agent **610** may modify webpage code **612** to form modified webpage code **612a** by identifying and modifying any portions of webpage code **612** that include scripts (e.g., those used for purposes of webpage analytics). For instance, agent **610** may identify a portion of webpage code **612** that include a JavaScript™ or other script through its inclusion of a <script> HTML element. For instance, a script associated with Google Analytics may look similar to the following:

---

```
<!-- Google Analytics -->
<script>
  (function(i,s,o,g,r,a,m){i['GoogleAnalyticsObject']=r;i[r]=i[r]||function(){
    (i[r].q=i[r].q||[]).push(arguments)},i[r].l=1*new Date();a=s.createElement(o),
    m=s.getElementsByTagName(o)[0];a.async=1;a.src=g;m.parentNode.insertBefore(a,m)}
    (window,document,'script','https://www.google-analytics.com/analytics.js','ga');
    ga('create', 'UA-XXXXX-Y', 'auto');
    ga('send', 'pageview');
  </script>
<!-- End Google Analytics -->
```

---

In such cases, agent 610 may modify webpage code 612 to remove the <script> tag and associated code, to completely disable the script and form modified webpage code 612a, in some embodiments.

In further embodiments, agent 610 may disable a script in webpage code 612 by simply disabling its synchronous operation. As would be appreciated, synchronous execution of a script can disrupt the rendering of the webpage by client 602, thereby increasing its page load time. Thus, rather than completely preventing execution of the script at all, agent 610 may instead opt to convert the script into asynchronous form in modified webpage code 612a. For instance, agent 610 may insert an “async” attribute into the script, such as in the following example:

```
<script async src="https://www.google-analytics.com/
analytics.js"></script>
```

Of course, agent 610 may always opt to leave existing web content in webpage code 612, in accordance with its policy. For instance, if the network traffic load is considered low and the page load times of the website are acceptable, agent 610 may instead opt to send webpage code 612 to client 602 without modification. In other embodiments, agent 610 may be responsible for inserting the web content into webpage code 612, prior to delivery to client 602.

In conclusion, the techniques herein introduce a web content management system that is able to make decisions on the fly based on a policy. Such a policy may be set so as to allow or block web content based on considerations such as performance, security, and/or other runtime metrics.

In closing, FIG. 7 illustrates an example simplified procedure for dynamic web content management, in accordance with one or more embodiments described herein. For example, a non-generic, specifically configured device (e.g., device 200) may perform procedure 2000 by executing stored instructions (e.g., illustrative application monitoring process 248). The procedure 700 may start at step 705, and continues to step 710, where, as described in greater detail above, an agent executed by the device may intercept webpage code for a website sent from an application server to a client of the website. In various embodiments, the agent may comprise a sidecar proxy, cloud access security broker (CASB), or may even be executed directly on the web application server that serves the webpage to the client.

At step 715, as detailed above, the agent may identify a portion of the webpage code as being used for webpage analytics. In some embodiments, the agent may do so by identifying a HyperText Markup Language (HTML) script tag in the portion of the webpage code. In further embodiments, the agent may do so by identifying an HTML metadata tag in the portion of the webpage code.

At step 720, the agent may form modified webpage code by disabling the portion of the webpage code, based on one or more performance metrics associated with the website, as described in greater detail above. In various embodiment, the one or more performance metrics may comprise a page load time, a network traffic load, or the like. In some embodiments, the agent may disable the portion of the webpage code by removing an HTML script tag from the webpage code. In further embodiments, the agent may disable immediate execution of a script in the portion of the webpage code by converting the script into an asynchronous script. In some embodiments, the agent may also form the modified webpage code based further in part on a location of the client, whether a user of the client is logged into an account associated with the website, or the like.

At step 725, as detailed above, the agent may send the modified webpage code to the client of the website. Since

the portion of the webpage code has been disabled (e.g., by completely disabling the analytics content inserted into the code or causing it to be executed asynchronously), this can help to improve performance and address any other concerns associated with the webpage analytics. Of course, this process may also be performed dynamically, such as by allowing or enabling the analytics content to be sent to clients under certain conditions. Procedure 700 then ends at step 730.

It should be noted that while certain steps within procedure 700 may be optional as described above, the steps shown in FIG. 7 are merely examples for illustration, and certain other steps may be included or excluded as desired. Further, while a particular order of the steps is shown, this ordering is merely illustrative, and any suitable arrangement of the steps may be utilized without departing from the scope of the embodiments herein.

While there have been shown and described illustrative embodiments above, it is to be understood that various other adaptations and modifications may be made within the scope of the embodiments herein. For example, while certain embodiments are described herein with respect to certain types of networks in particular, the techniques are not limited as such and may be used with any computer network, generally, in other embodiments. Moreover, while specific technologies, protocols, and associated devices have been shown, such as Java, TCP, IP, and so on, other suitable technologies, protocols, and associated devices may be used in accordance with the techniques described above. For example, while certain terms have been used herein, such as tiers and nodes, other protocols may use other terms while still remaining within the scope of the present disclosure, such as process groups (instead of tiers), split process groups due to host grouping (instead of tiers with longer names, such as “EAST-tomcat-server”), and so on. In addition, while certain devices are shown, and with certain functionality being performed on certain devices, other suitable devices and process locations may be used, accordingly. That is, the embodiments have been shown and described herein with relation to specific network configurations (orientations, topologies, protocols, terminology, processing locations, etc.). However, the embodiments in their broader sense are not as limited, and may, in fact, be used with other types of networks, protocols, and configurations.

Moreover, while the present disclosure contains many other specifics, these should not be construed as limitations on the scope of any embodiment or of what may be claimed, but rather as descriptions of features that may be specific to particular embodiments of particular embodiments. Certain features that are described in this document in the context of separate embodiments can also be implemented in combination in a single embodiment. Conversely, various features that are described in the context of a single embodiment can also be implemented in multiple embodiments separately or in any suitable sub-combination. Further, although features may be described above as acting in certain combinations and even initially claimed as such, one or more features from a claimed combination can in some cases be excised from the combination, and the claimed combination may be directed to a sub-combination or variation of a sub-combination.

For instance, while certain aspects of the present disclosure are described in terms of being performed “by a server” or “by a controller,” those skilled in the art will appreciate that agents of the application intelligence platform (e.g., application agents, network agents, language agents, etc.) may be considered to be extensions of the server (or con-

## 21

troller) operation, and as such, any process step performed “by a server” need not be limited to local processing on a specific server device, unless otherwise specifically noted as such. Furthermore, while certain aspects are described as being performed “by an agent” or by particular types of agents (e.g., application agents, network agents, etc.), the techniques may be generally applied to any suitable software/hardware configuration (libraries, modules, etc.) as part of an apparatus or otherwise.

Similarly, while operations are depicted in the drawings in a particular order, this should not be understood as requiring that such operations be performed in the particular order shown or in sequential order, or that all illustrated operations be performed, to achieve desirable results. Moreover, the separation of various system components in the embodiments described in the present disclosure should not be understood as requiring such separation in all embodiments.

The foregoing description has been directed to specific embodiments. It will be apparent, however, that other variations and modifications may be made to the described embodiments, with the attainment of some or all of their advantages. For instance, it is expressly contemplated that the components and/or elements described herein can be implemented as software being stored on a tangible (non-transitory) computer-readable medium (e.g., disks/CDs/RAM/EEPROM/etc.) having program instructions executing on a computer, hardware, firmware, or a combination thereof. Accordingly, this description is to be taken only by way of example and not to otherwise limit the scope of the embodiments herein. Therefore, it is the object of the appended claims to cover all such variations and modifications as come within the true intent and scope of the embodiments herein.

What is claimed is:

1. A method, comprising:
  - intercepting, by an agent executed by a device, webpage code for a website sent from an application server to a client of the website;
  - identifying, by the agent, a portion of the webpage code as being used for webpage analytics by identifying a HyperText Markup Language metadata tag in the portion of the webpage code used by a search engine to perform analytics on traffic to the website;
  - determining, by the agent and based on a policy, that the webpage code poses a security risk by identifying sensitive data or vulnerable security data in the webpage code as defined in the policy, wherein the policy is set by default or specified via a user interface;
  - forming, by the agent and responsive to determining that the webpage code poses the security risk, a modified webpage code by disabling the portion of the webpage code, wherein the agent disables the portion of the webpage code by removing the HyperText Markup Language metadata tag from the webpage code; and
  - sending, by the agent, the modified webpage code to the client of the website.
2. The method as in claim 1, wherein the policy comprises one or more performance metrics that include a page load time of the website.
3. The method as in claim 1, wherein identifying the portion of the webpage code as being used for webpage analytics further comprises:
  - identifying a HyperText Markup Language script tag in the portion of the webpage code.
4. The method as in claim 3, wherein forming the modified webpage code by disabling the portion of the webpage code further comprises:

## 22

removing the HyperText Markup Language script tag from the webpage code.

5. The method as in claim 3, wherein forming the modified webpage code by disabling the portion of the webpage code further comprises:

disabling immediate execution of a script in the portion of the webpage code by converting the script into an asynchronous script.

6. The method as in claim 1, wherein the policy comprises one or more performance metrics that include network traffic load of a network associated with the website.

7. The method as in claim 1, wherein the agent comprises a sidecar proxy or cloud access security broker.

8. The method as in claim 1, wherein the policy further defines whether the client is a bot, whether the client is logged into an account associated with the website, an identity of a user of the client, or a location of the client.

9. An apparatus, comprising:

one or more network interfaces;

a processor coupled to the one or more network interfaces and configured to execute one or more processes; and a memory configured to store a process that is executable by the processor, the process when executed configured to:

intercept webpage code for a website sent from an application server to a client of the website;

identify a portion of the webpage code as being used for webpage analytics by identifying a HyperText Markup Language metadata tag in the portion of the webpage code used by a search engine to perform analytics on traffic to the website;

determine, based on a policy, that the webpage code poses a security risk by identifying sensitive data or vulnerable security data in the webpage code as defined in the policy, wherein the policy is set by default or specified via a user interface;

form, responsive to determining that the webpage code poses a security risk, a modified webpage code by disabling the portion of the webpage code, wherein the apparatus disables the portion of the webpage code by removing the HyperText Markup Language metadata tag from the webpage code; and

send the modified webpage code to the client of the website.

10. The apparatus as in claim 9, wherein the policy comprises one or more performance metrics that include a page load time of the website.

11. The apparatus as in claim 9, wherein the apparatus further identifies the portion of the webpage code as being used for webpage analytics by:

identifying a HyperText Markup Language script tag in the portion of the webpage code.

12. The apparatus as in claim 11, wherein the apparatus further forms the modified webpage code by disabling the portion of the webpage code by:

removing the HyperText Markup Language script tag from the webpage code.

13. The apparatus as in claim 11, wherein the apparatus further forms the modified webpage code by disabling the portion of the webpage code by:

disabling immediate execution of a script in the portion of the webpage code by converting the script into an asynchronous script.

14. The apparatus as in claim 9, wherein the policy comprises one or more performance metrics that include network traffic load of a network associated with the website.

## 23

15. The apparatus as in claim 9, wherein the policy further defines whether the client is a bot, whether the client is logged into an account associated with the website, an identity of a user of the client, or a location of the client.

16. A tangible, non-transitory, computer-readable medium storing program instructions that cause an agent of a device to execute a process comprising:

intercepting webpage code for a website sent from an application server to a client of the website;

identifying a portion of the webpage code as being used for webpage analytics by identifying a HyperText Markup Language metadata tag in the portion of the webpage code used by a search engine to perform analytics on traffic to the website;

determining, by the agent and based on a policy, that the webpage code poses a security risk by identifying sensitive data or vulnerable security data in the webpage code as defined in the policy, wherein the policy is set by default or specified via a user interface; forming, by the agent and responsive to determining that the webpage code poses the security risk, a modified

## 24

webpage code by disabling the portion of the webpage code, wherein the agent disables the portion of the webpage code by removing the HyperText Markup Language metadata tag from the webpage code; and sending the modified webpage code to the client of the website.

17. The tangible, non-transitory, computer-readable medium as in claim 16, wherein the policy comprises one or more performance metrics that include a page load time of the website.

18. The tangible, non-transitory, computer-readable medium as in claim 16, wherein the policy comprises one or more performance metrics that include network traffic load of a network associated with the website.

19. The tangible, non-transitory, computer-readable medium as in claim 16, wherein the policy further defines whether the client is a bot, whether the client is logged into an account associated with the website, an identity of a user of the client, or a location of the client.

\* \* \* \* \*